Evaluación del módulo #6, #7 y #8

Consigna del proyecto 👨🏻‍💻

alkemy

Evaluación del módulo

Proyecto: Node & Express Web App

Situación inicial 📍

Unidad solicitante: Departamento de Desarrollo Backend de una empresa de tecnología.

Una reconocida empresa tecnológica necesita **desarrollar una aplicación web** para **gestión de usuarios y datos**. El equipo de arquitectura ha definido las bases del proyecto, pero requiere un **desarrollador backend** para llevarlo a producción.

Has sido seleccionado para este desafío. Tu misión será construir, iterar y demostrar el funcionamiento de una aplicación real utilizando **Node.js** y **Express**, aplicando conocimientos adquiridos en los **módulos 6, 7 y 8** del programa.

El proyecto deberá abordar, en **tres etapas**, los aspectos clave del desarrollo backend moderno: desde servir contenido web y gestionar rutas, hasta conectar con una base de datos, implementar un ORM, proteger rutas con JWT y exponer una API RESTful profesional.

Nuestro objetivo 📋

Desarrollar de manera individual una **aplicación web funcional** con **Node.js** y **Express**, que:

- Sirva contenido web dinámico.
- Gestione usuarios y datos persistidos en base de datos.
- Exponga una API RESTful con rutas protegidas mediante **JWT**.
- Integre persistencia con archivos planos, ORM para manejo de relaciones y funcionalidades clave como subida de archivos.

La implementación será **progresiva**, en tres partes alineadas a los módulos del curso:

→ **Módulo 6**: estructura inicial del servidor, rutas, vistas y persistencia básica.

→ **Módulo 7**: integración con base de datos, modelos y operaciones CRUD usando ORM.

→ **Módulo 8**: diseño de una API RESTful segura, con subida de archivos y autenticación.

Requerimientos ✅

Estos son los fundamentos que deberás respetar y aplicar durante todo el desarrollo del proyecto. Funcionan como condiciones base:

- La aplicación deberá estar desarrollada en **Node.js** (v18 o superior) utilizando el framework **Express.js**.
- El proyecto debe estar estructurado en **rutas**, **controladores**, **middlewares** y **servicios** siguiendo una arquitectura modular.
- Se espera el uso de **Sequelize** o **Mongoose** como ORM, según el tipo de base de datos seleccionada.
- La persistencia debe implementarse sobre una base de datos relacional (**PostgreSQL**) o documental (**MongoDB**).
- El repositorio del proyecto debe estar subido a **GitHub**, con estructura clara y un archivo `README.md` que documente su uso e instalación.
- Se deberá incluir al menos un mecanismo básico de seguridad utilizando **JSON Web Tokens (JWT)**.

Requerimientos específicos

Estos son los **elementos concretos** que la aplicación desarrollada deberá contemplar **funcional y técnicamente**. Serán validados en cada una de las entregas.

→ Gestión general de la aplicación

- Servir contenido web estático o dinámico desde **Express.**
- Implementar **rutas públicas y privadas** (protegidas con JWT).
- Permitir **autenticación de usuarios** (login / registro).
- Aplicar **lógica de persistencia** en archivos planos para funcionalidades simples (logs, configuración, etc.).
- Permitir **ejecución del servidor desde línea de comandos** con nodemon o scripts.

→ Acceso y gestión de datos

- Conectar la aplicación a una **base de datos real** (MongoDB o PostgreSQL).
- **Modelar y relacionar entidades** utilizando un ORM (relaciones 1:1, 1:N y N:M).
- Permitir **operaciones CRUD completas** sobre al menos dos entidades clave.
- Implementar **consultas filtradas y búsquedas** dinámicas.
- Aplicar **manejo de errores y validaciones** en las operaciones con base de datos.

→ Seguridad y API

- Crear una **API RESTful** con rutas protegidas mediante autenticación JWT.
- Implementar **subida de archivos** (por ejemplo, imágenes de usuario).
- Validar **tipos de archivo y tamaño** en el endpoint de subida.
- Estructurar las respuestas de la API con formato consistente (`status, message, data`).

Paso a paso 👣

Parte 1 – Módulo 6: Primeros pasos con Node y Express

Este proyecto será avanzado paso a paso en cada clase y podrás completarlo progresivamente a través de los espacios ***asincrónicos.*** También tendrás espacios de consulta en las clases sincrónicas para despejar tus dudas.

Todo gran desarrollo backend comienza con una base sólida. En esta primera parte del desafío, deberás levantar los cimientos de tu aplicación. Como responsable técnico del proyecto, tu rol será **iniciar el servidor con Node.js y Express**, configurar la estructura de carpetas, implementar las primeras rutas y servir contenido web.

Esta etapa es clave: lo que construyas aquí se convertirá en la base del sistema que más adelante integrará una base de datos y expondrá una API segura. Por eso, además de que el servidor funcione, tu objetivo será dejar un entorno bien organizado, claro y modular.

1. **Conociendo Node y Express** (Lección 1)
   - Objetivo: Comprender el entorno Node.js y el rol de Express en el desarrollo backend moderno.
   - Tareas:
     → Investigar el ecosistema Node.js y para qué se utiliza.
     → Identificar qué aporta Express sobre Node puro.
     → Realizar un **esquema visual del flujo** básico servidor–cliente.
   - Tarea PLUS (opcional):
     → Crear una infografía personal del stack técnico del proyecto.
2. **Instalación y configuración de Node** (Lección 2)
   - Objetivo: Preparar el entorno de desarrollo y configurar el proyecto base.
   - Tareas:
     → Instalar Node.js correctamente (mínimo versión 18).
     → Inicializar el proyecto con `npm init` y completar todos los campos relevantes del `package.json`.

→ Crear el archivo principal `index.js` o `app.js`, con una función que imprima "Servidor iniciado".

- Criterio de cumplimiento:

  → El proyecto debe ejecutarse sin errores desde terminal usando node `index.js`.
  → Se debe justificar brevemente en el README por qué se eligió `index.js` o `app.js` como nombre base del archivo principal.

- Tarea PLUS:

  → Configurar uso de variables de entorno (`dotenv`) para el puerto del servidor.

3. **Gestión de paquetes en Node** (Lección 3)

- Objetivo**:** Incorporar y configurar paquetes clave para el desarrollo backend.

- Tareas:

  → Instalar y declarar las siguientes dependencias:

    - `express` (requerido)
    - `dotenv` (recomendado)
    - `nodemon` como `devDependency`

  → Crear scripts personalizados en `package.json`:

    - `npm start`
    - `npm run dev`

  → Agregar comentarios en el código para describir qué hace cada línea relevante.

- Criterio de cumplimiento:

  → Los scripts deben ejecutarse correctamente.
  → El README debe incluir instrucciones claras de instalación y ejecución.
  → Justificar por qué se eligieron esos scripts (si se cambió el nombre, explicar la decisión).

4. **Sirviendo contenido web** (Lección 4)

   - Objetivo: Crear las primeras rutas del servidor y servir contenido al usuario.

   - Tareas:

     → Crear **al menos 2 rutas públicas** (`/` y `/status`), cada una con respuestas en HTML o JSON.
     → Configurar la carpeta `/public` para servir **al menos 1 archivo** estático.
     → Agregar middleware `express.static()` correctamente.

   - Criterio de cumplimiento:

     → La aplicación debe servir contenido estático desde `/public`.
     → Al menos **una ruta** debe devolver respuesta en JSON y otra en HTML.
     → Justificar en el README si se decide no utilizar `/public`, explicando la alternativa elegida (por ejemplo, motor de plantillas).

   - Tarea PLUS (opcional):

     → Implementar **una vista dinámic**a con `ejs` u otro motor de plantillas.

5. **Persistencia en archivos planos** (Lección 5)

   - Objetivo: Implementar almacenamiento simple de datos usando el sistema de archivos.

   - Tareas:

     → Crear un archivo `log.txt` y una función que registre en él cada visita a una ruta específica.
     → Usar `fs.appendFile()` para agregar líneas de texto.
     → Validar que el texto registrado tenga la siguiente estructura mínima: fecha, hora, ruta accedida.

   - Criterio de cumplimiento:

     → El archivo debe registrar **al menos 3 accesos simulados**.

→ Justificar si se elige otro tipo de evento para registrar (por ejemplo, errores o inicio del servidor).

6. **Ejecución de un aplicativo Node** (Lección 6)

- Objetivo: Validar que el servidor funcione correctamente y documentar la estructura del proyecto.
- Tareas:
  - → Ejecutar la aplicación con npm run dev.
  - → Validar el acceso a las rutas creadas.
  - → Crear y completar el archivo README.md con:
    - Requisitos del sistema
    - Instrucciones de instalación
    - Ejemplos de uso
  - → Subir el repositorio a GitHub con nombre claro y estructura organizada.
- Criterio de cumplimiento:
  - → El repositorio debe incluir al menos 5 carpetas bien nombradas (routes, controllers, middlewares, public, logs).
  - → Justificar en el README si se decide una estructura distinta (por ejemplo, incluir /utils, /services, etc.).
- Tarea PLUS (opcional):
  - → Crear un router.js externo y conectar las rutas con app.use().

✅ Entregables

Para la presentación de esta primera parte del proyecto, deberás compartir los siguientes elementos:

1. **Repositorio en GitHub**
   - Estructura clara del proyecto (carpetas, archivos fuente y `README.md`).
   - `Commit` inicial y `commits` parciales que den cuenta del desarrollo.
   - Scripts de ejecución funcionales: `npm run dev`, `npm start`.

2. **Carpeta en Google Drive compartida**
    - Nombre sugerido: *TP Integrador JS - [Tu Nombre y Apellido]*
    - Subcarpeta para la Parte 1: *Parte 1 – Módulo 6*
    - Contenidos mínimos:
        - Capturas de pantalla del servidor en funcionamiento.
        - Capturas del uso de rutas (`/`, `/status`) y archivos planos (`log.txt`).
        - (Opcional) Captura del archivo de vistas si se utilizó motor de plantillas.
        - Documento breve con reflexiones técnicas (puede ser parte del README o un archivo `.md`/`.pdf` por separado).

Esta carpeta también se usará para compartir las entregas de los módulos 7 y 8, por lo que es importante que esté bien organizada desde el inicio.

🔍 ¿Qué vamos a validar?

Durante la evaluación de esta primera entrega del proyecto, se valorará especialmente que lo desarrollado **cumpla con los objetivos de la consigna y se evidencie un uso correcto y reflexivo de los conocimientos adquiridos.** Los criterios de evaluación específicos que serán considerados en la revisión son:

★ Aplicación de requerimientos técnicos

- ☐ Uso correcto de Node.js y Express para levantar el servidor.
- ☐ Gestión adecuada de paquetes con npm (express, nodemon, dotenv).
- ☐ Configuración funcional del archivo principal (index.js o app.js).
- ☐ Estructura de proyecto organizada en carpetas (routes, controllers, middlewares, etc.).
- ☐ Rutas funcionales y middleware express.static() correctamente aplicado.
- ☐ Registro de datos en archivos planos mediante el módulo fs.

Se espera que el entregable funcione correctamente y que cumpla con las instrucciones técnicas mínimas detalladas en las tareas. Se valorará la robustez y la claridad de la estructura.

★ Objetivo de la consigna

- ☐ La solución presentada cumple con el propósito de esta primera parte: *servir contenido web y estructurar la aplicación base para el backend.*
- ☐ Se valida que el servidor sea accesible, que las rutas funcionen, y que el proyecto quede listo para la integración con base de datos en la siguiente parte.
- ☐ Las decisiones tomadas deben tener coherencia con el objetivo del módulo y anticipar la escalabilidad del proyecto.

Se valorará que el trabajo esté enfocado, bien resuelto y alineado al desafío planteado, sin desvíos innecesarios.

★ Justificación y reflexión

- ☐ Se espera registro de las decisiones tomadas en el código y en el README.md.
- ☐ Deben explicarse, cuando aplique, los motivos de ciertas elecciones: *nombre del archivo principal, estructura de carpetas, uso o no de plantillas, etc.*
- ☐ Se valora el pensamiento reflexivo sobre el diseño del backend y su proyección futura.

Las justificaciones no deben ser extensas, pero sí claras y técnicas.

★ Claridad y organización

- ☐ El código debe estar ordenado, indentado y comentado correctamente.
- ☐ Las carpetas deben tener nombres representativos y seguir convenciones del ecosistema Node.js.
- ☐ El README.md debe incluir instrucciones precisas y estar bien redactado.
- ☐ El entregable debe poder entenderse y ejecutarse fácilmente por otra persona que acceda al repositorio.

💼 Portafolio

Esta primera entrega representa la base del backend de una aplicación web profesional. Es una excelente oportunidad para comenzar a construir un caso sólido para tu portafolio, destacando tu capacidad para:

- Levantar y estructurar un servidor en Node.js con buenas prácticas.
- Servir contenido dinámico o estático a través de Express.
- Implementar lógica de persistencia básica y modularizar rutas.
- Documentar correctamente un proyecto técnico real en GitHub.

Aunque la app aún no tiene base de datos ni API, este primer paso ya demuestra dominio del entorno backend y es un diferencial valioso en perfiles trainee.

¡Éxitos!

Nos vemos más adelante

alkemy